

LOW-LEVEL DESIGN (LLD)

Project Title:

RAG-Based Customer Support Assistant using LangGraph with HITL

INTRODUCTION

The Low-Level Design (LLD) provides a comprehensive and detailed blueprint of how the proposed system is implemented at an internal and functional level. It focuses on translating the high-level architecture into clearly defined modules, specifying the responsibilities, interactions, and data flow between each component. By breaking down the system into smaller, manageable units, the LLD ensures a clear understanding of how each part contributes to the overall functionality of the RAG-based customer support assistant.

This design covers essential aspects such as module-level architecture, data structures, workflow execution, and conditional decision-making logic. Each module, including document processing, chunking, embedding, retrieval, query handling, and response generation, is described in detail to illustrate how data is processed step-by-step. The LLD also defines how the system leverages embeddings and vector databases to perform efficient similarity search, and how the retrieved context is utilized by the Large Language Model (LLM) to generate accurate and context-aware responses.

Overall, the Low-Level Design serves as a detailed implementation guide that bridges the gap between conceptual design and practical development. It promotes modularity, scalability, and maintainability, enabling developers to efficiently build, test, and enhance the system. By clearly defining each component and its role, the LLD ensures that the RAG-based customer support assistant can be implemented in a structured, reliable, and extensible manner.

MODULE-LEVEL DESIGN

Document Processing Module

This module is responsible for loading and extracting text from PDF documents. It uses document loaders to convert unstructured PDF content into raw text format, which is then passed to the chunking module.

Chunking Module

The chunking module divides the extracted text into smaller segments based on a predefined chunk size and overlap. This ensures that each chunk retains sufficient contextual information while remaining suitable for embedding and retrieval.

Embedding Module

This module converts each text chunk into a numerical vector representation using an embedding model. These embeddings capture semantic relationships and are essential for performing similarity-based retrieval.

Vector Storage Module

The vector storage module stores the generated embeddings along with their corresponding text chunks in ChromaDB. It enables efficient indexing and retrieval of data based on similarity search.

Retrieval Module

This module processes user queries by converting them into embeddings and retrieving the top-k most relevant chunks from the vector database. It ensures that only the most contextually relevant information is passed to the LLM.

Query Processing Module

The query processing module combines the user query with the retrieved chunks to form a structured prompt. This prompt is then passed to the LLM for response generation.

Graph Execution Module (LangGraph)

This module manages the workflow of the system using a graph-based structure. It defines nodes, edges, and state transitions, enabling dynamic control flow and conditional routing based on system outputs.

Human-in-the-Loop (HITL) Module

This module handles cases where the system cannot confidently generate a response. It forwards the query and context to a human agent and integrates the human response back into the system output.

DATA STRUCTURES

- **Document Representation**

```
{  
  "doc_id": "string",  
  "content": "full_text"  
}
```

- **Chunk Representation**

```
{  
  "chunk_id": "string",  
  "text": "chunk_text",  
  "metadata": {}  
}
```

- **Embedding Structure**

embedding = [float, float, float, ...]

- **Query-Response Schema**

```
{  
  "query": "user_input",  
  "retrieved_chunks": [],  
  "answer": "generated_response",  
  "confidence": 0.0,  
  "route": "answer | escalate"  
}
```

- **State Object (LangGraph)**

```
state = {  
  "query": str,  
  "retrieved_chunks": list,  
  "answer": str,  
  "confidence": float,  
  "route": str  
}
```

WORKFLOW DESIGN (LangGraph)

Nodes

- Input Node: Accepts user query
- Processing Node: Handles retrieval + LLM
- Output Node: Returns final response

Edges

- Input → Processing
- Processing → Decision
- Decision → Output / HITL

State Flow

The state object is a central data structure that is passed between nodes in the workflow and updated at each stage of execution. It acts as a shared container that maintains all relevant information required for processing the user query throughout the system. This includes the original query, retrieved document chunks, generated response, confidence score, and the final routing decision. This approach enables efficient tracking of intermediate results, supports conditional decision-making, and allows the system to dynamically adapt its execution flow based on the current state.

CONDITIONAL ROUTING LOGIC

The system uses decision logic to determine whether to respond directly or escalate.

Escalation Criteria:

- Confidence score below threshold (e.g., < 0.7)
- No relevant chunks retrieved
- Query is out-of-scope
- Query is too complex

Direct Response Criteria:

- Relevant chunks found
- High confidence score
- Clear and complete answer generated

HUMAN-IN-THE-LOOP (HITL) DESIGN

Trigger Conditions

- **Low confidence response:** When the system generates an answer with a confidence score below a predefined threshold, indicating that the response may not be reliable or accurate enough to present to the user.
- **Retrieval failure:** When the system is unable to find relevant chunks from the vector database, or the retrieved results do not provide sufficient context to generate a meaningful answer.
- **Ambiguous or complex queries:** When the user query is unclear, lacks sufficient detail, or involves multi-step reasoning that exceeds the system's capability, making it difficult to produce a precise and accurate response.

Flow After Escalation

- **Query and context forwarded to human agent:** When escalation is triggered, the system sends the user query along with the retrieved contextual information to a human agent for further analysis.
- **Human reviews and generates response:** The human agent evaluates the query and the provided context, and then formulates an accurate and appropriate response based on their understanding and expertise.
- **Response delivered to the user:** The human-generated response is returned through the system interface to the user, ensuring clarity, correctness, and reliability.
- **Optional storage for future improvement:** The interaction, including the query and human response, can be stored in the system to improve future performance, support learning mechanisms, or enhance the knowledge base.

API / INTERFACE DESIGN

- **Input Format**

The system accepts user queries in a structured format to ensure consistency in processing. The input typically consists of a single query string provided by the user through the interface. This standardized format allows the query to be easily passed through different modules of the system for embedding, retrieval, and response generation.

```
{  
  "query": "What is the refund policy?"  
}
```

- **Output Format**

The system returns the response in a structured format that includes not only the generated answer but also additional metadata. This may include a confidence score indicating the reliability of the response and the source of the information, such as the document or human intervention. This structured output improves transparency and helps in decision-making for further processing.

```
{  
  "answer": "Refund is allowed within 7 days...",  
  "confidence": 0.85,  
  "source": "document"  
}
```

- **Interaction Flow**

The interaction flow represents the step-by-step process through which a user query is handled by the system. Initially, the user submits a query through the interface, which is then processed by the system. The query is converted into an embedding and used to retrieve relevant chunks from the vector database. These retrieved chunks are passed to the LLM, which generates a context-aware response. Based on the confidence and completeness of the response, the system either returns the answer directly to the user or routes the query to the Human-in-the-Loop (HITL) module for further handling. This structured flow ensures efficient processing, accurate responses, and reliable fallback mechanisms.

[User → System → Retrieval → LLM → Response / HITL]

ERROR HANDLING

No relevant chunks found: If the retrieval module fails to identify any relevant chunks from the vector database, the system returns a fallback response indicating that sufficient information is not available. Optionally, the query can be escalated to the Human-in-the-Loop (HITL) module for further assistance.

LLM failure: In cases where the Large Language Model fails to generate a response due to API errors, timeouts, or unexpected issues, the system attempts a retry mechanism. If the issue persists, the query is escalated to ensure the user still receives a response.

Invalid input: If the user provides incomplete, empty, or improperly formatted input, the system validates the request and returns an appropriate error message, prompting the user to provide a valid query.

System failure: For unexpected system-level failures, such as crashes or internal errors, the system logs the error for debugging and monitoring purposes and notifies the user with a generic error message while maintaining system stability.

IMPLEMENTATION NOTES

Use of LangChain: LangChain is utilized to simplify the integration of various pipeline components such as document loading, text splitting, embedding generation, and retrieval, reducing development complexity.

Use of ChromaDB: ChromaDB is employed as the vector database for storing embeddings and enabling efficient similarity search, which is essential for retrieval-based operations.

Use of LangGraph: LangGraph is used for workflow orchestration, allowing the system to follow a structured, stateful execution model with conditional routing and decision-making capabilities.

Modular architecture: The system is designed with a modular structure, where each component operates independently. This improves scalability, maintainability, and ease of future enhancements.

Extensibility: The design allows easy integration of new embedding models, LLMs, or additional workflow nodes without requiring major changes to the existing system.